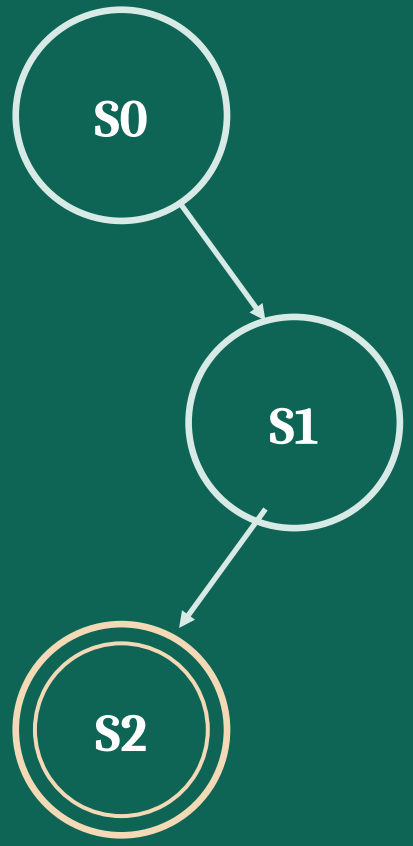
Informatik · Zusatz · Level 16 · Welt D: Sprachdesign & Ausblick

Ausblick: Die Turingmaschine

Aufgabenbörse · Band, Kopf und Übergänge · Church-Turing-These und Halteproblem · Abschluss der Einheit



Kernfrage

Was kann eine Maschine, die auf ihr Band schreiben darf – und gibt es Grenzen, die keine Maschine überwindet?

Einstieg



Die Aufgabenbörse

Aufgabenbörse: Löst die Aufgabe eines anderen Teams

20 min | Teams | Aufgaben aus Level 15, AB16

1. Jedes Team erhält die **Kurzversion** der Aufgabe eines anderen Teams.
2. Löst sie wie im Abitur: **Operatoren** beachten, Notation einhalten, eigenes Blatt.
3. Nach 15 Minuten: Erwartungshorizont vom Autorenteam holen und **vergleichen**.
4. **Rückmeldung** an das Autorenteam (AB16, Aufgabe 1a): Was war klar? Was fehlte?

Erarbeitung



Wo der Keller an seine Grenze kommt



Kann ein Kellerautomat die Sprache $a^n b^n c^n$ erkennen?

$a^n b^n$ war kein Problem: Für jedes a ein A auf den Keller, für jedes b eines herunter. Und jetzt kommen noch genauso viele c …

Der Keller wird beim Vergleichen verbraucht

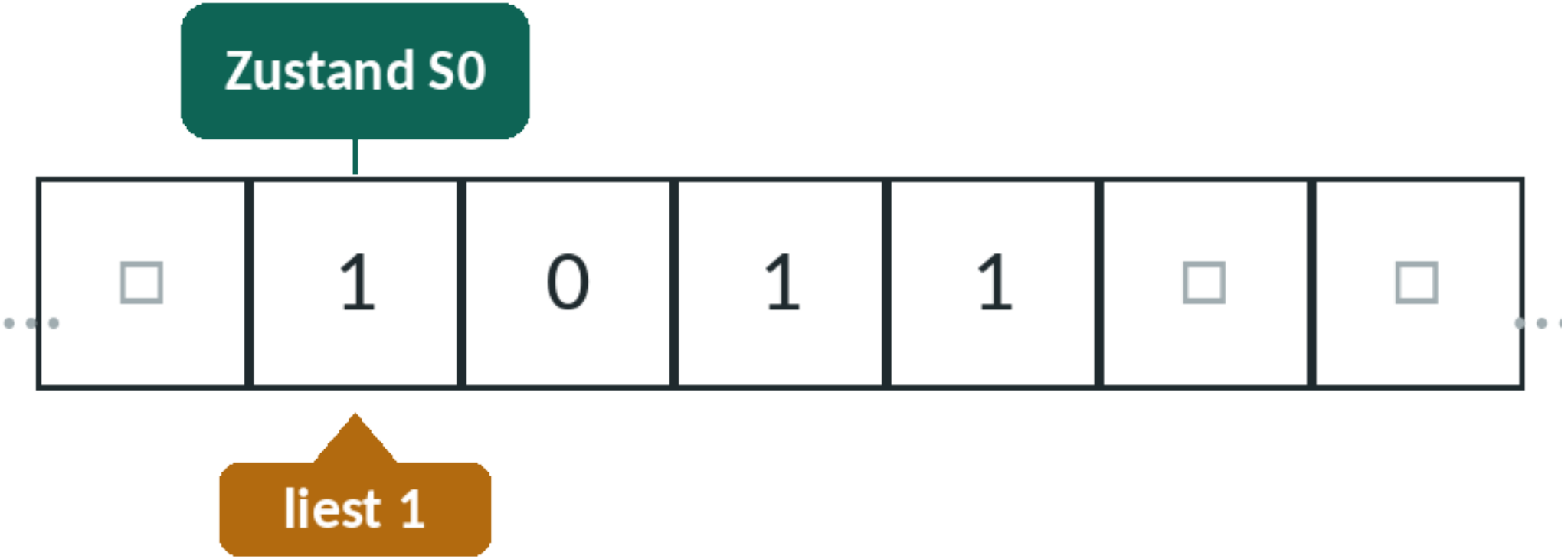
- Nach den b ist der Keller **leer** – die Information „wie viele a?“ ist weg
- Zwei A pro a? Beim ersten b weiß der Automat nicht, **wann die b aufhören**
- Anschaulich: **Ein Keller kann nur einmal abgezählt werden** (strenger Beweis: Universität)
- Was fehlt, ist eine Maschine, die ihre Eingabe **verändern** darf und beliebig oft **hin- und herläuft**
- → Alan Turing, 1936: die **Turingmaschine**

Erarbeitung



Die Turingmaschine

Aufbau: Band, Schreib-Lese-Kopf, Zustand



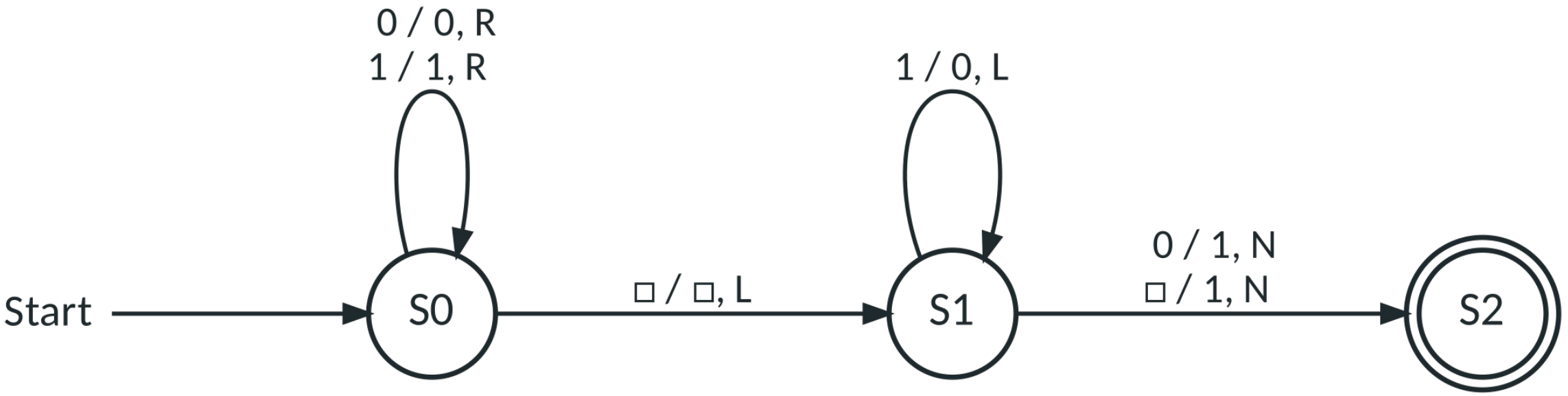
Das Band ist beidseitig unbegrenzt, leere Felder enthalten das Blank □. Der Kopf liest ein Feld, schreibt und bewegt sich um ein Feld.

Ein Übergang hat drei Teile

- **(Zustand, gelesenes Zeichen) → (Folgezustand, geschriebenes Zeichen, Bewegung)**
- Bewegung: **L** (links), **R** (rechts), **N** (bleibt stehen)
- Kantenbeschriftung im Graphen: „gelesen / geschrieben, Bewegung“, z. B. **1 / 0, L**
- Kein passender Übergang → die Maschine **hält**
- Hält sie in einem **Endzustand**, ist die Eingabe akzeptiert; der Bandinhalt ist die **Ausgabe**
- Start: Zustand S0, Kopf auf dem ersten Zeichen der Eingabe

Beispiel: Binärzahl + 1

Σ = {0, 1} · S0: ans Ende laufen · S1: Übertrag nach links tragen · S2: fertig



Eingabe 1011 (= 11) → Ausgabe 1100 (= 12). Was passiert bei 111?

Unplugged: Du bist der Schreib-Lese-Kopf

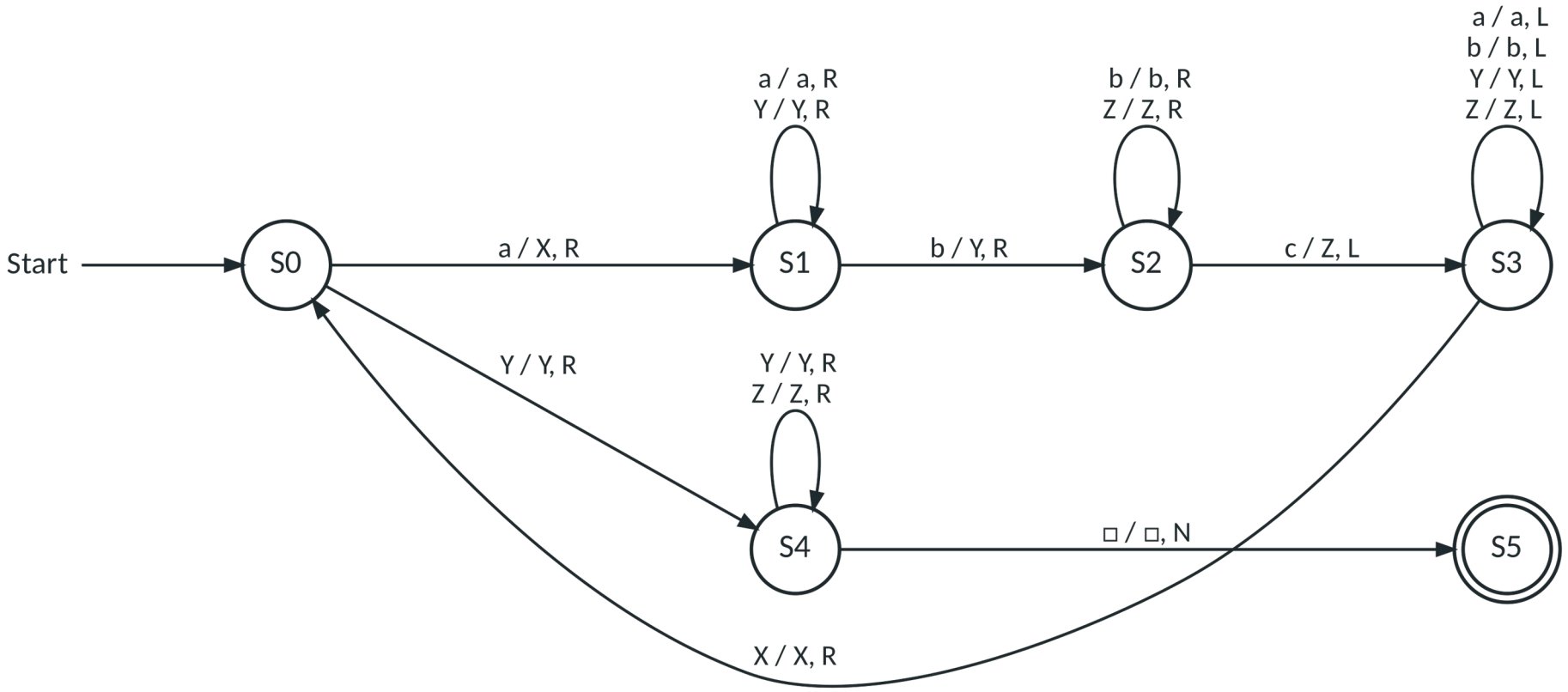
15 min | Tandem | Papierstreifen-Band, Zustands- und Regelkarten, AB16

1. **Kopf:** liest das Feld unter der Münze vor, radiert, schreibt, schiebt die Münze.
2. **Steuerung:** sucht die Regel auf der Regelkarte, sagt sie an, wechselt die Zustandskarte.
3. Nach jedem Schritt **Rollentausch**; Schritte zählen.
4. „Binärzahl + 1“ für **1101** ausführen (AB16, Aufgabe 3a), Ergebnis dezimal prüfen.
5. Dann: eigene Maschine „jedes a wird zu b“ entwerfen und testen (3b).

AutomatenLab
Turing-Labor

$a^n b^n c^n$: Markieren, zurücklaufen, wiederholen

Pro Runde: ein a → X, ein b → Y, ein c → Z, zurück zum letzten X. Sind alle a verbraucht (S0 liest Y), prüft S4, ob nur noch Y und Z folgen.



Das Band ist das Gedächtnis: Markierungen bleiben erhalten und können beliebig oft gelesen werden.

Rätsel: Der fleißige Biber



Die Frage

Welche Maschine mit n Zuständen schreibt auf leerem Band die meisten Einsen – und hält trotzdem?



Bekannt

- BB(1) = 1, BB(2) = 4
- BB(3) = 6, BB(4) = 13
- BB(5) = 4098 (bewiesen 2024)



Unbekannt

BB(6) ≥ 10 ↑ ↑ 15 – eine Zahl, die man nicht mehr ausschreiben kann. Die Biber-Funktion wächst schneller als jede berechenbare Funktion.

Church-Turing-These

- **Alles, was ein Algorithmus berechnen kann, kann eine Turingmaschine berechnen.**
- Jeder Computer, jede Programmiersprache ist **genauso mächtig** wie dieses einfache Modell – nicht mächtiger
- Eine **These**, kein Satz: nicht beweisbar, aber seit 1936 in jedem Fall bestätigt
- Deshalb ist die Turingmaschine **das** Modell, um über Berechenbarkeit zu sprechen
  - Chomsky-Hierarchie: Typ 3 DEA · Typ 2 NKA · Typ 0 Turingmaschine

Das Halteproblem – eine Geschichte

- Eine Firma verspricht das Programm **HALT**: Für jedes Programm P sagt es korrekt „hält“ oder „läuft ewig“
- Eine Informatikerin baut **TROTZ**: Es fragt HALT „Halte ich selbst?“ – und tut das **Gegenteil**
- Sagt HALT „hält“ → TROTZ läuft ewig. Sagt HALT „läuft ewig“ → TROTZ hält sofort
- HALT irrt sich in jedem Fall → **HALT kann nicht existieren**
- Alan Turing, 1936: **Kein Algorithmus** entscheidet für alle Programme, ob sie halten
- Es gibt Grenzen, die keine Maschine überwindet – egal wie schnell

Turing-Labor



AutomatenLab

Turing-Labor

Maschinen Schritt für Schritt ausführen: **Binärzahl + 1**, **aⁿbⁿcⁿ**, **Fleißiger Biber**. Band beobachten, eigene Eingaben testen. Challenge: die fehlende Regel ergänzen.

`https://automatenlab.frankjuchim.de/#/turing?programm=binaer-plus-1`

Abschluss: Quiz, Lernlandkarte, Feedback

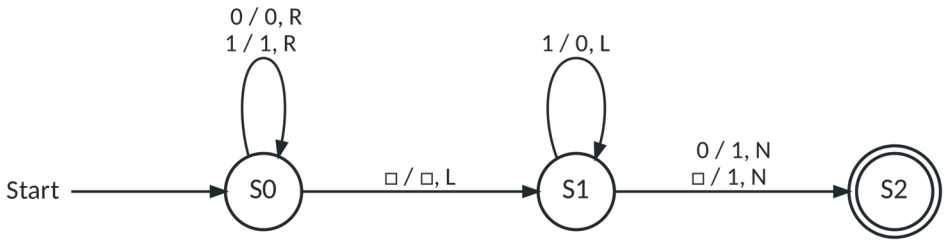
20 min | Einzelarbeit | App, Lernlandkarte, AB16

1. **Level-Quiz Welt D** in der App (8 min).
2. **Lernlandkarte:** jedes Level markieren – sicher / unsicher / noch üben; für jedes „noch üben“ eine konkrete Übung notieren (7 min).
3. **Feedback** auf AB16: Was hat geholfen? Was anders? Welche Frage ist offen? (5 min)

AutomatenLab
Level-Quiz

Merke: Turingmaschine

- Band + Schreib-Lese-Kopf + endlich viele Zustände; Übergang **(Zustand, gelesen) → (Zustand, geschrieben, L/R/N)**
- Sie darf **schreiben** und **hin- und herlaufen** – deshalb schafft sie $a^n b^n c^n$, woran der Keller scheitert
- **Church-Turing-These:** Jeder Algorithmus ist eine Turingmaschine
- **Halteproblem:** Kein Algorithmus entscheidet für alle Programme, ob sie halten



Was ihr aus dem AutomatenLab mitnehmt

- Welt A: Automaten analysieren, entwickeln, minimieren, übersetzen (Mealy)
- Welt B: Grenzen begründen, Kellerautomaten dokumentieren
- Welt C: Grammatiken entwerfen, regulär ↔ kontextfrei, Chomsky-Hierarchie
- Welt D: eigene Sprachen entwerfen – und die Turingmaschine als Ausblick

Hausaufgabe
Übungsklausur im Abi-Format (135 min, 45 BE) bearbeiten und mit dem Erwartungshorizont kontrollieren. Merkblatt M12 „Abi-Spickzettel“ nutzen.